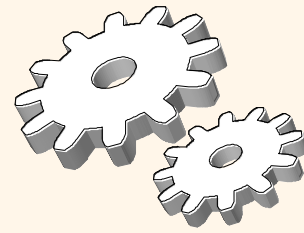


Storing Data: Disks and Files

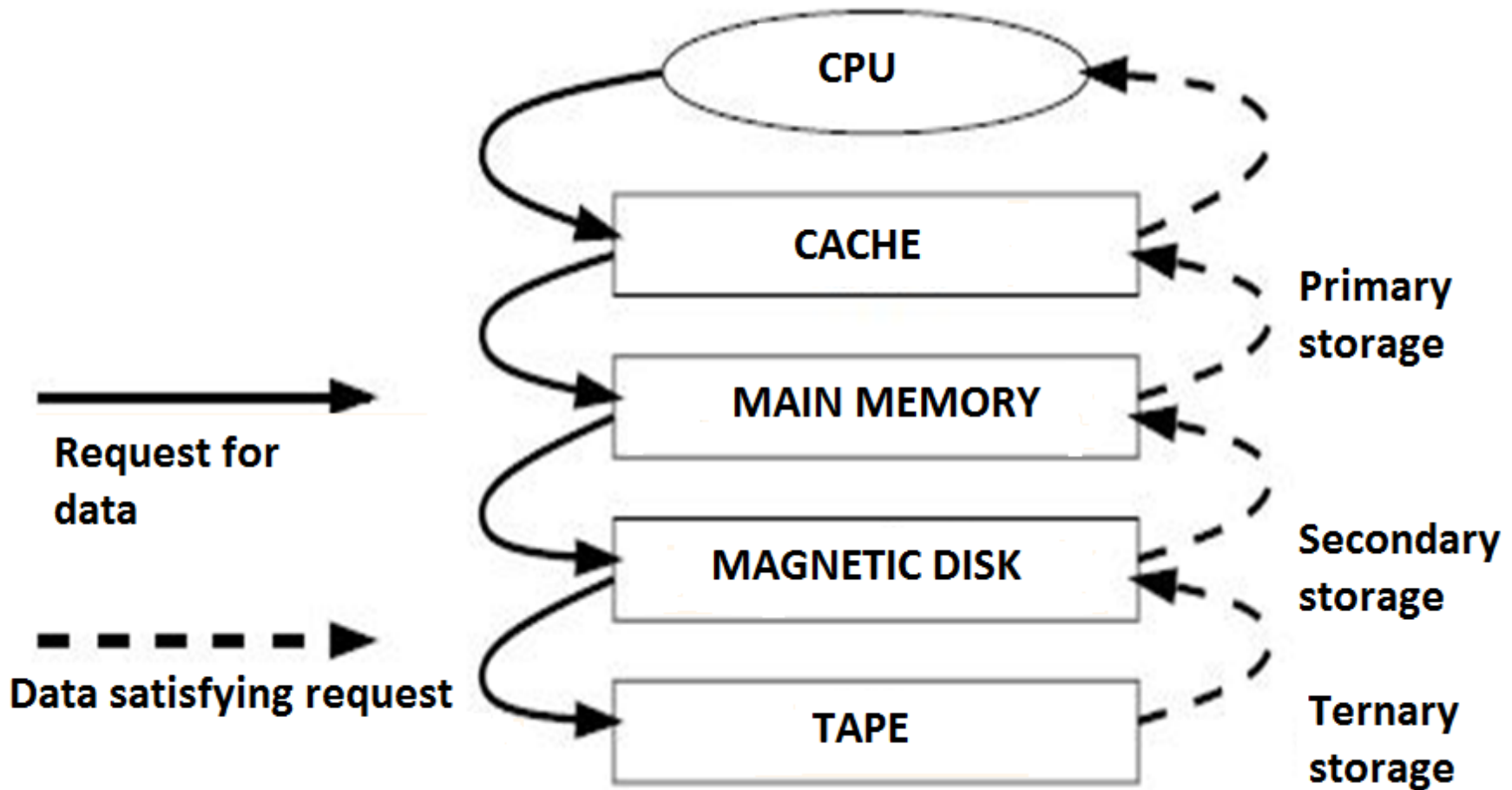
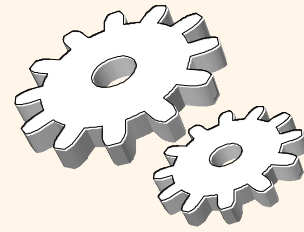
Chapter 9

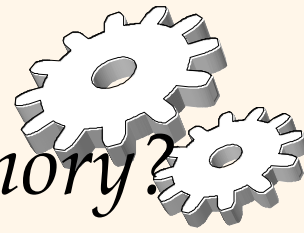


Disks and Files

- ❖ DBMS stores information on (“hard”) disks.
- ❖ This has major implications for DBMS design!
 - **READ:** transfer data from disk to main memory (RAM).
 - **WRITE:** transfer data from RAM to disk.
 - Both are high-cost operations, relative to in-memory operations, so must be planned carefully!

Memory Hierarchy

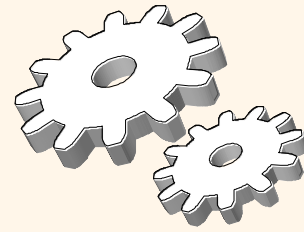




Why Not Store Everything in Main Memory?

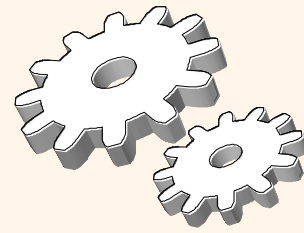
- ❖ *Costs too much.* \$1000 will buy you either 128MB of RAM or 7.5GB of disk (in 2003).
- ❖ in 2017: 16GB of RAM is 750 TL, 10TB of HDD is 2079 TL, 1TB of SSD is 1809 TL
- ❖ *Main memory is volatile.* We want data to be saved between runs. (Obviously!)
- ❖ Typical storage hierarchy:
 - Main memory (RAM) for currently used data.
 - Disk for the main database (secondary storage).
 - Tapes for archiving older versions of the data (tertiary storage).

Comparison of Storage Devices



Type	Capacity	Access Time	Price
Main memory	256 MB	5×10^{-7} seconds	0.2 \$/MB
Hard disk	60 GB	15×10^{-3} seconds	3 \$ /GB
Floppy disk	1.44 MB	1×10^{-1} seconds	0.2 \$ / MB
CD-ROM	650 MB	75 milli seconds	0.002 \$ / MB
DVD	4.7 GB	112 milli seconds	0.001 \$ / MB

These values belong to year 2003, but the above table can give you an idea to make a comparison between storage devices.

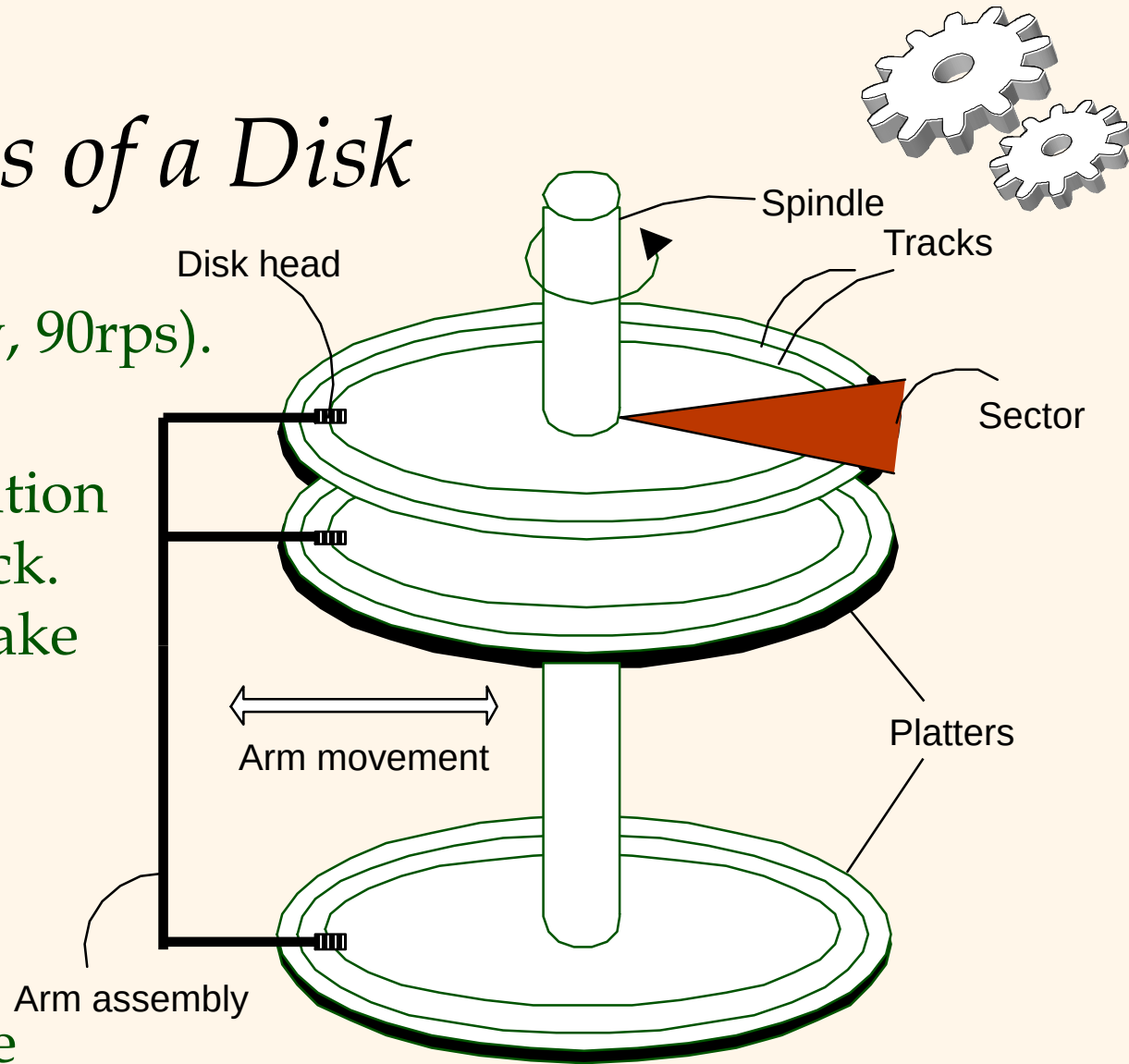


Disks

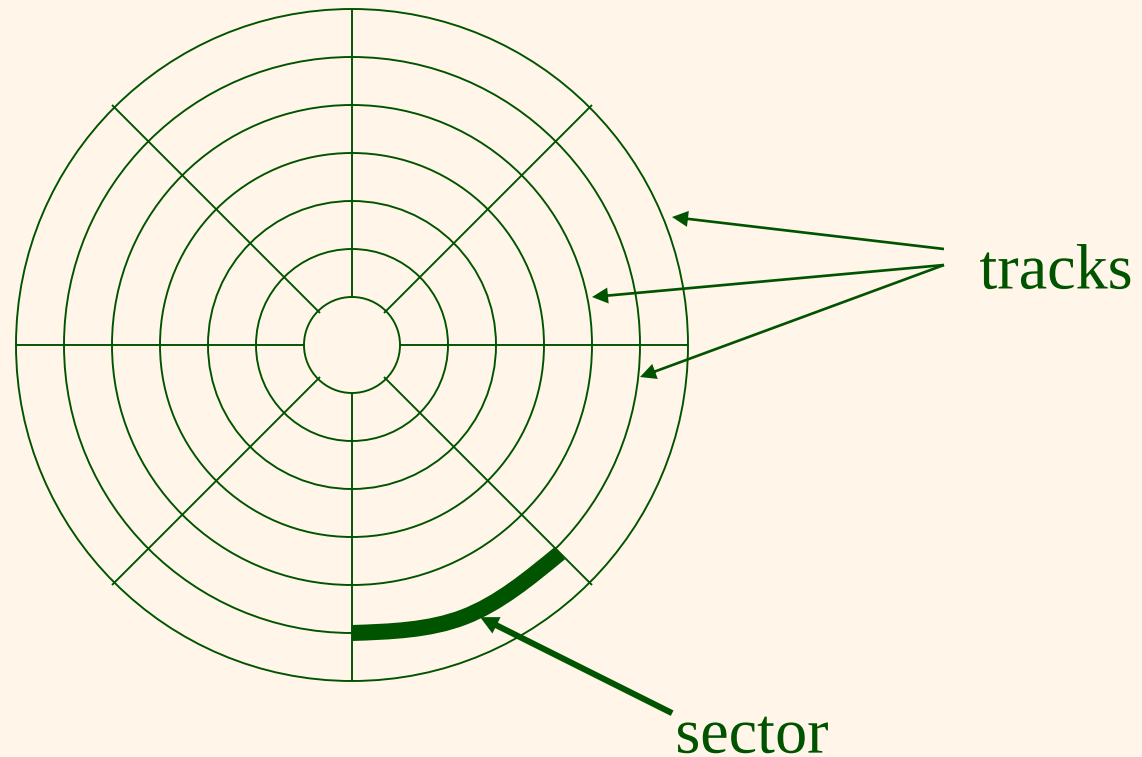
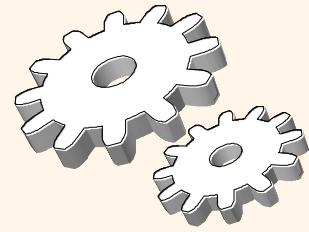
- ❖ Secondary storage device of choice.
- ❖ Main advantage over tapes: random access vs. *sequential*.
- ❖ Data is stored and retrieved in units called *disk blocks* or *pages*.
- ❖ Unlike RAM, time to retrieve a disk page varies depending upon location on disk.
 - Therefore, relative placement of pages on disk has major impact on DBMS performance!

Components of a Disk

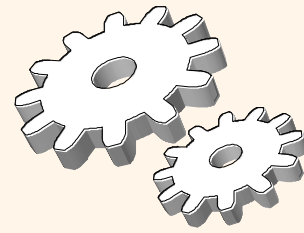
- ❖ The platters spin (say, 90rps).
- ❖ The arm assembly is moved in or out to position a head on a desired track. Tracks under heads make a *cylinder* (imaginary!).
- ❖ Only one head reads/writes at any one time.
- ❖ *Block size* is a multiple of *sector size* (which is fixed).



Looking at a surface

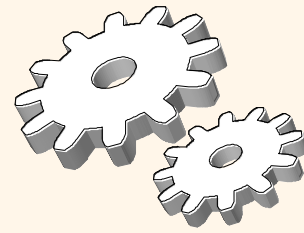


Surface of disk showing tracks and sectors



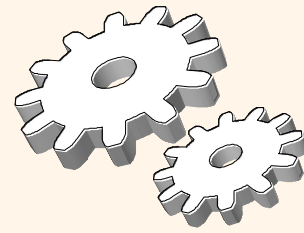
Organization of Disks

- ❖ Disk contains concentric **tracks**.
- ❖ Tracks are divided into **sectors**
- ❖ A **sector** is the smallest addressable unit in a disk.



Accessing Data

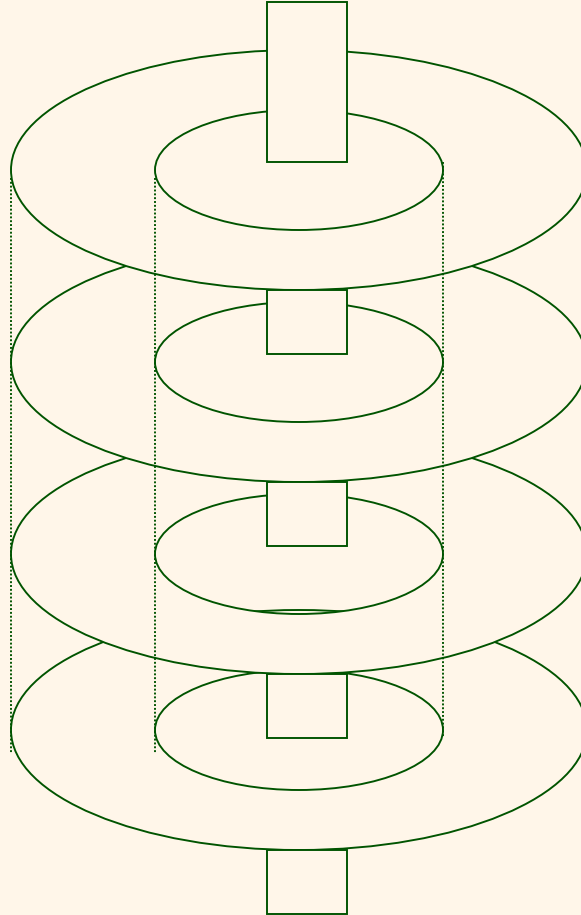
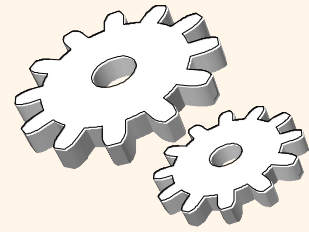
- ❖ When a program reads a byte from the disk, the operating system locates the surface, track and sector containing that byte, and reads the entire sector into a special area in main memory called **buffer**.
- ❖ The bottleneck of a disk access is moving the read/write arm. So it makes sense to store a file in tracks that are below/above each other in different surfaces, rather than in several tracks in the same surface.

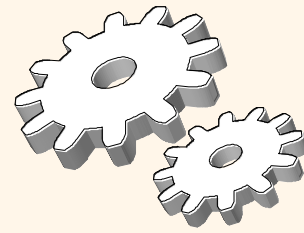


Cylinders

- ❖ A **cylinder** is the set of tracks at a given radius of a disk pack.
 - i.e. a cylinder is the set of tracks that can be accessed without moving the disk arm.
- ❖ All the information on a cylinder can be accessed without moving the read/write arm.

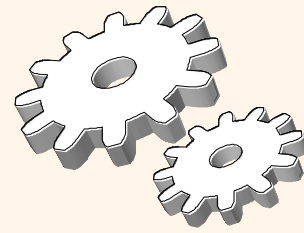
Cylinders





Estimating Capacities

- ❖ Track capacity = # of sectors per track * bytes per sector
- ❖ Cylinder capacity = # of tracks per cylinder * track capacity
- ❖ Drive capacity = # of cylinders * cylinder capacity
- ❖ Number of cylinders = # of tracks in a surface



Exercise

❖ Store a file of 20000 records on a disk with the following characteristics:

of bytes per sector = 512

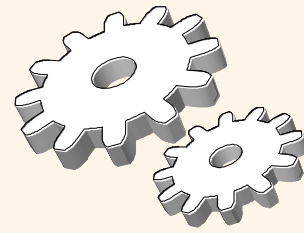
of sectors per track = 40

of tracks per cylinder = 12

of cylinders = 1331

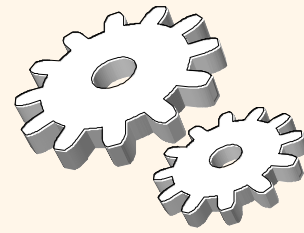
Q1. How many cylinders does the file require if each data record requires 256 bytes?

Q2. What is the total capacity of the disk?



Organizing Tracks by Block

- ❖ Disk tracks may be divided into user-defined blocks rather than into sectors.
- ❖ Blocks can be fixed or variable length.
- ❖ A block is usually organized to hold an integral number of logical records.
- ❖ **Blocking Factor** = number of records stored in a block.
- ❖ No internal fragmentation, no record spanning two blocks.
- ❖ In block-addressing scheme each block of data is accompanied by one or more *subblocks* containing extra information about the block.



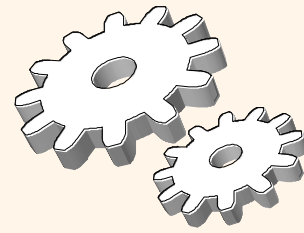
Accessing a Disk Page (Block)

❖ The time to access a sector in a track on a surface is divided into 3 components

1. Seek Time: Time to move the read/write arm to the correct cylinder

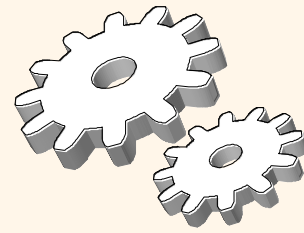
2. Rotational delay (or latency): Time it takes for the disk to rotate so that the desired block is under the read/write head

3. Transfer time: Once the read/write head is positioned over the data, this is the time it takes for transferring data



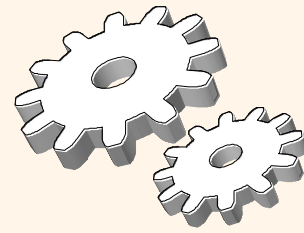
Arranging Pages on Disk

- ❖ *'Next'* block concept:
 - blocks on same track, followed by
 - blocks on same cylinder, followed by
 - blocks on adjacent cylinder
- ❖ Blocks in a file should be arranged sequentially on disk (by *'next'*), to minimize seek and rotational delay.
- ❖ For a *sequential scan*, *pre-fetching* several pages at a time is a big win!



Disk Access Time

- ❖ Time to read/write 1 block of data from/to disk is computed as follows:
- ❖ Access time = seek time +
rotational delay +
transfer time



Accessing N Blocks on a Disk

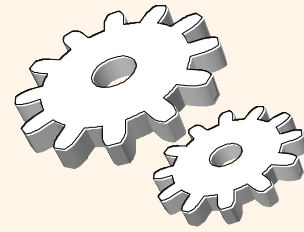
- ❖ Time required to access N blocks which reside next to each other on a disk is computed as

Seek time + rotational delay + N * transfer time

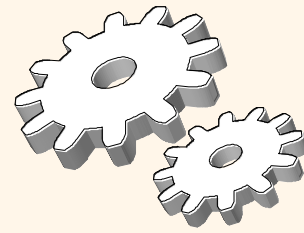
- ❖ Time required to access N randomly distributed blocks on a disk is computed as

N * (Seek time + rotational delay + transfer time)

Exercise

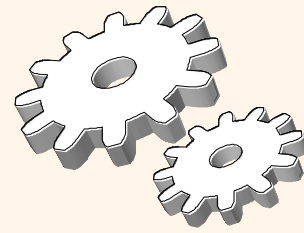


- ❖ Consider a disk with a sector size of 512 bytes, 2000 tracks per surface, 50 sectors per track, five double-sided platters, and average seek time of 10 msec.
 1. What is the capacity of a track in bytes?
 2. What is the capacity of each surface?
 3. What is the capacity of the disk?
 4. How many cylinders does the disk have?
 5. Give examples of valid block sizes. Is 256 bytes a valid block size? 2048? 51200?
 6. If the disk platters rotate at 5400 rpm, what is the maximum rotational delay?
 7. If one track of data can be transferred per revolution, what is the transfer rate?



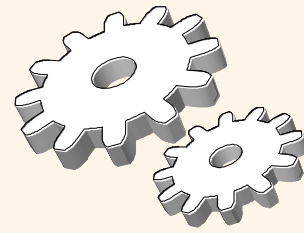
RAID

- ❖ Disk Array: Arrangement of several disks that gives abstraction of a single, large disk.
- ❖ Goals: Increase performance and reliability.
- ❖ Two main techniques:
 - **Data striping:** Data is partitioned; size of a partition is called the striping unit. Partitions are distributed over several disks.
 - **Redundancy:** More disks => more failures. Redundant information allows reconstruction of data if a disk fails.



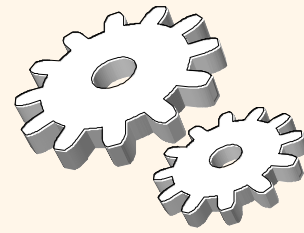
RAID Levels

- ❖ **Level 0:** No redundancy
- ❖ **Level 1:** Mirrored (two identical copies)
 - Each disk has a mirror image (check disk)
 - Parallel reads, a write involves two disks.
 - Maximum transfer rate = transfer rate of one disk
- ❖ **Level 0+1:** Striping and Mirroring
 - Parallel reads, a write involves two disks.
 - Maximum transfer rate = aggregate bandwidth



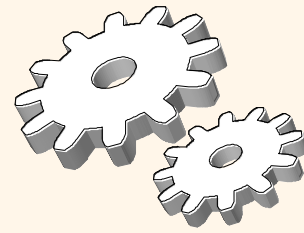
RAID Levels (Contd.)

- ❖ **Level 3: Bit-Interleaved Parity**
 - Striping Unit: One bit. One check disk.
 - Each read and write request involves all disks; disk array can process one request at a time.
- ❖ **Level 4: Block-Interleaved Parity**
 - Striping Unit: One disk block. One check disk.
 - Parallel reads possible for small requests, large requests can utilize full bandwidth
 - Writes involve modified block and check disk
- ❖ **Level 5: Block-Interleaved Distributed Parity**
 - Similar to RAID Level 4, but parity blocks are distributed over all disks



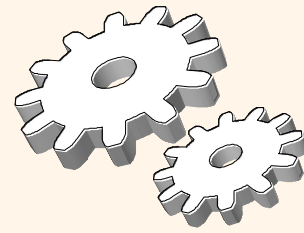
Disk Space Management

- ❖ Lowest layer of DBMS software manages space on disk.
- ❖ Higher levels call upon this layer to:
 - allocate/de-allocate a page
 - read/write a page
- ❖ Request for a *sequence* of pages must be satisfied by allocating the pages sequentially on disk! Higher levels don't need to know how this is done, or how free space is managed.



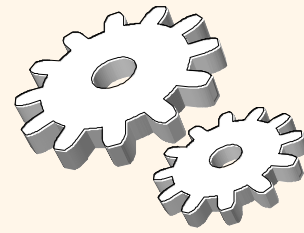
When a Page is Requested ...

- ❖ If requested page is not in pool:
 - Choose a frame for *replacement*
 - If frame is dirty, write it to disk
 - Read requested page into chosen frame
- ❖ *Pin* the page and return its address.
- * *If requests can be predicted (e.g., sequential scans) pages can be pre-fetched several pages at a time!*



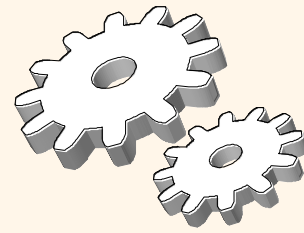
More on Buffer Management

- ❖ Requestor of page must unpin it, and indicate whether page has been modified:
 - *dirty* bit is used for this.
- ❖ Page in pool may be requested many times,
 - a *pin count* is used. A page is a candidate for replacement iff *pin count* = 0.
- ❖ CC & recovery may entail additional I/O when a frame is chosen for replacement.
(*Write-Ahead Log* protocol; more later.)



Buffer Replacement Policy

- ❖ Frame is chosen for replacement by a *replacement policy*:
 - Least-recently-used (LRU), Clock, MRU etc.
- ❖ Policy can have big impact on # of I/O's; depends on the *access pattern*.
- ❖ Sequential flooding: Nasty situation caused by LRU + repeated sequential scans.
 - # buffer frames < # pages in file means each page request causes an I/O. MRU much better in this situation (but not in all situations, of course).

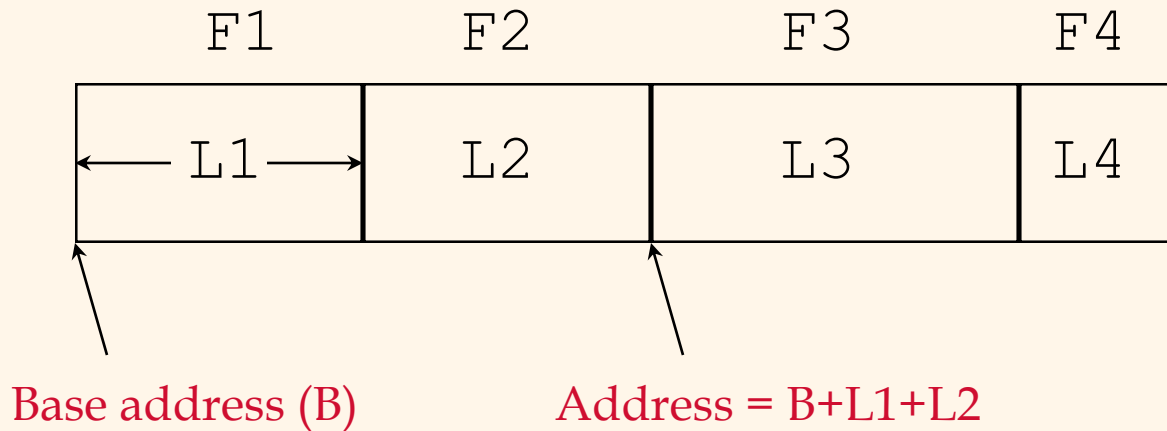
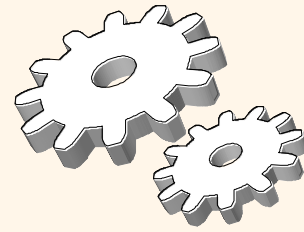


DBMS vs. OS File System

OS does disk space & buffer mgmt: why not let OS manage these tasks?

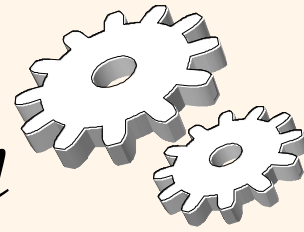
- ❖ Differences in OS support: portability issues
- ❖ Some limitations, e.g., files can't span disks.
- ❖ Buffer management in DBMS requires ability to:
 - *pin a page* in buffer pool, *force a page* to disk (important for implementing CC & recovery),
 - adjust *replacement policy*, and *pre-fetch pages* based on access patterns in typical DB operations.

Record Formats: Fixed Length

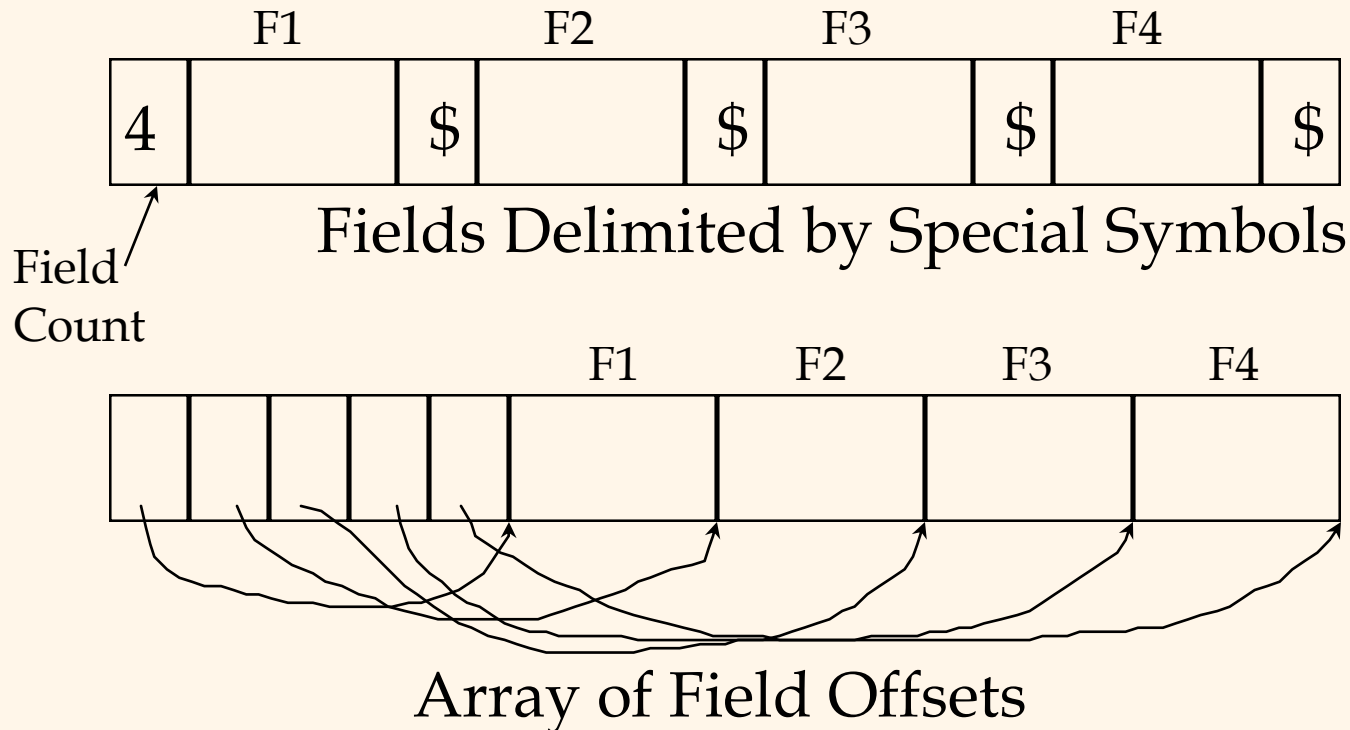


- ❖ Information about field types same for all records in a file; stored in *system catalogs*.
- ❖ Finding *i*'th field does not require scan of record.

Record Formats: Variable Length

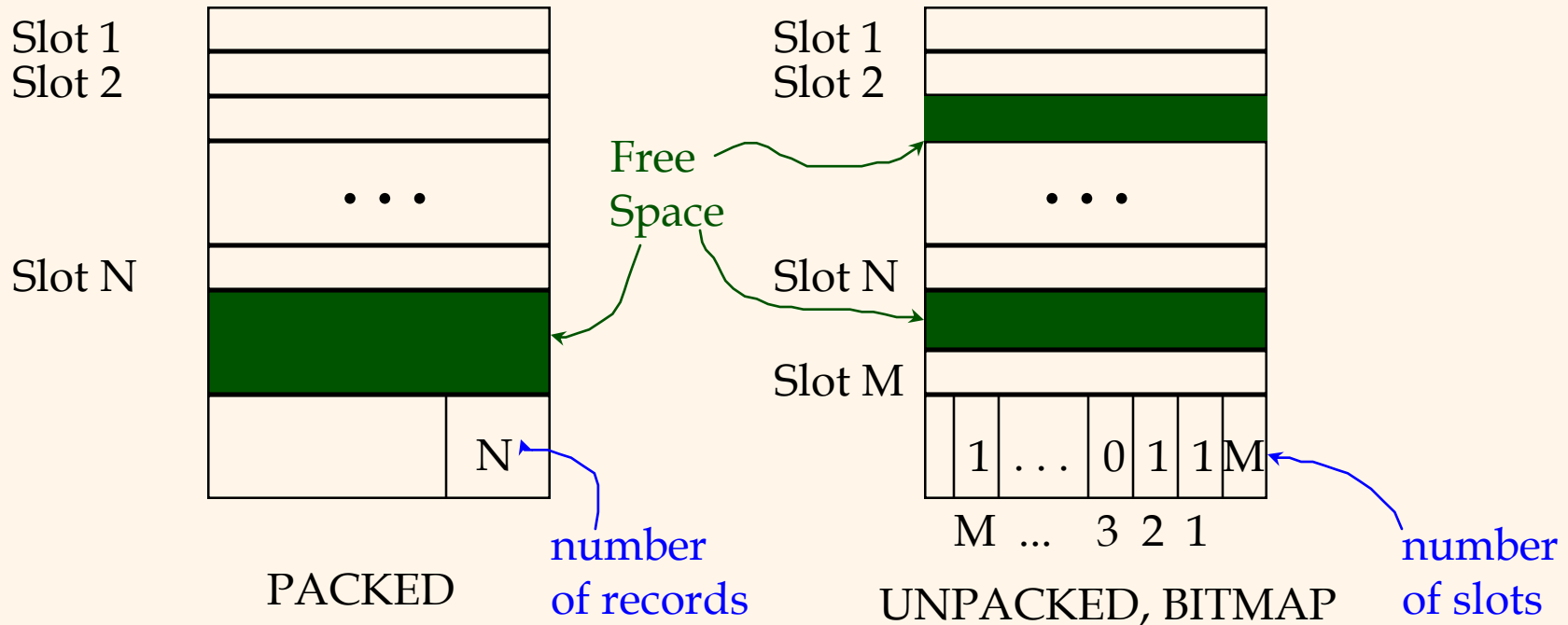
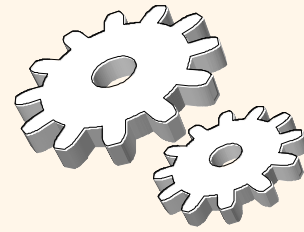


❖ Two alternative formats (# fields is fixed):



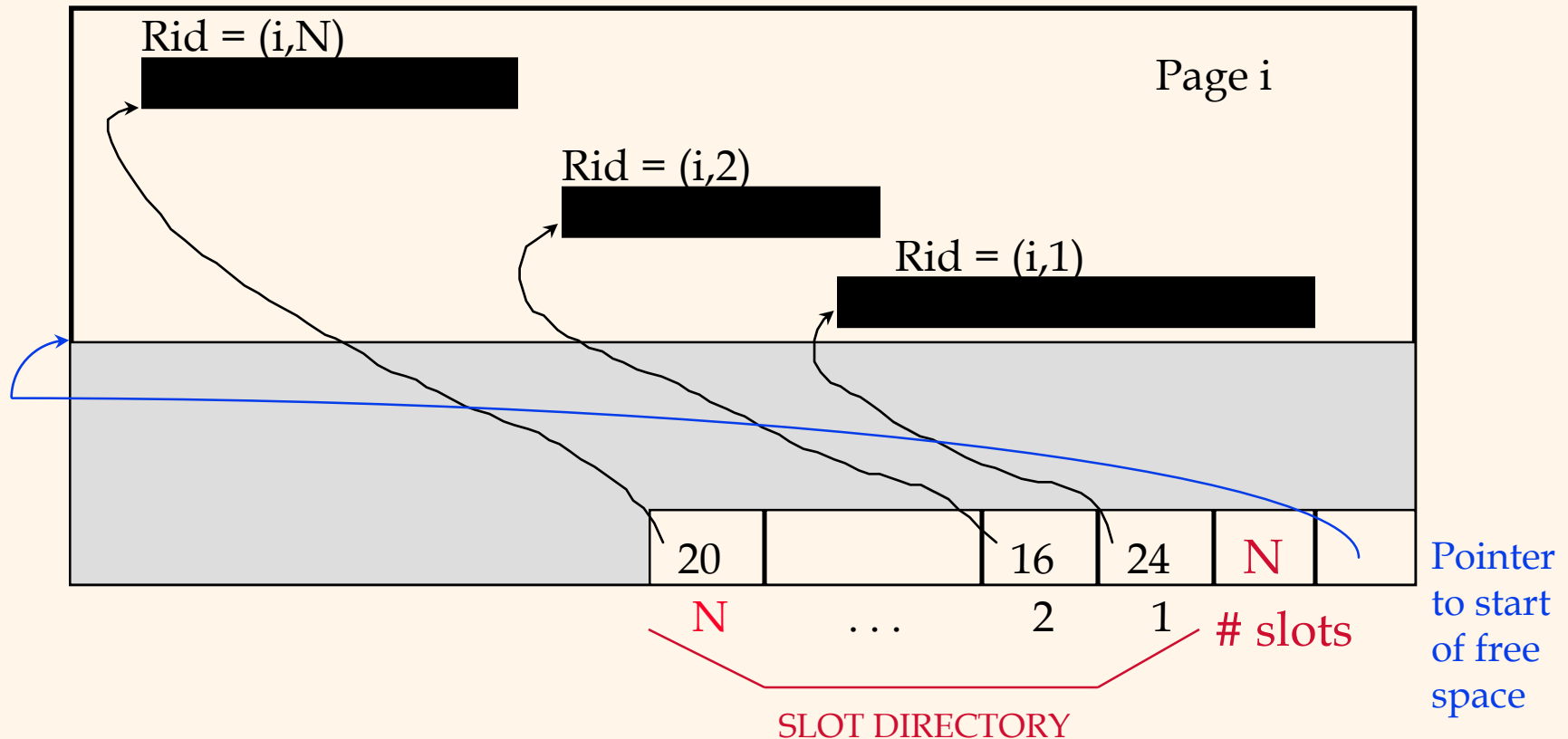
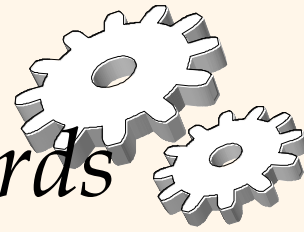
* Second offers direct access to i 'th field, efficient storage of nulls (special *don't know* value); small directory overhead.

Page Formats: Fixed Length Records

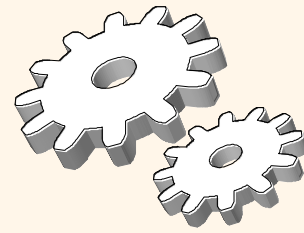


- * Record id = <page id, slot #>. In first alternative, moving records for free space management changes rid; may not be acceptable.

Page Formats: Variable Length Records

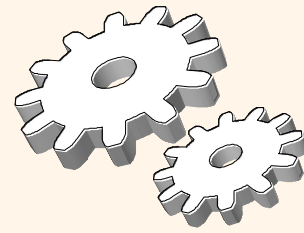


- * *Can move records on page without changing rid; so, attractive for fixed-length records too.*



Files of Records

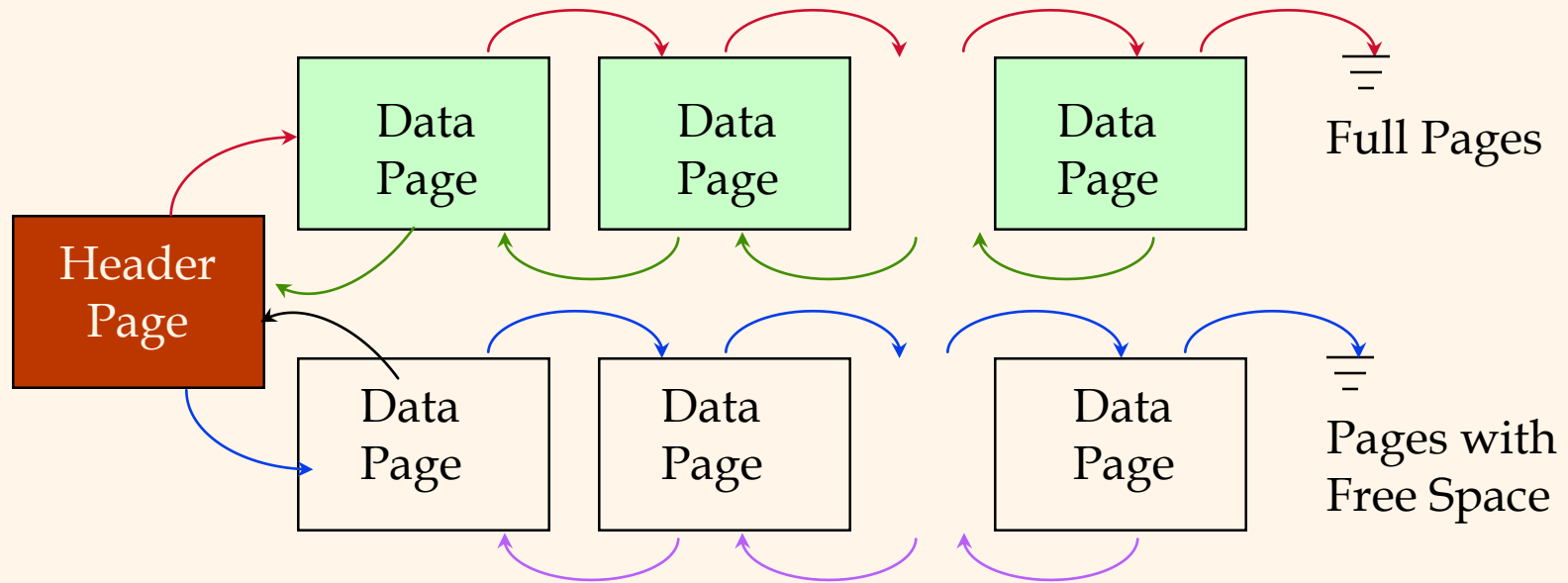
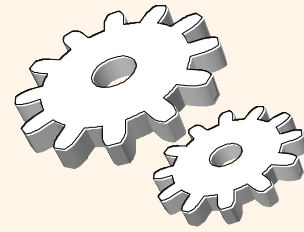
- ❖ Page or block is OK when doing I/O, but higher levels of DBMS operate on *records*, and *files of records*.
- ❖ FILE: A collection of pages, each containing a collection of records. Must support:
 - insert/delete/modify record
 - read a particular record (specified using *record id*)
 - scan all records (possibly with some conditions on the records to be retrieved)



Unordered (Heap) Files

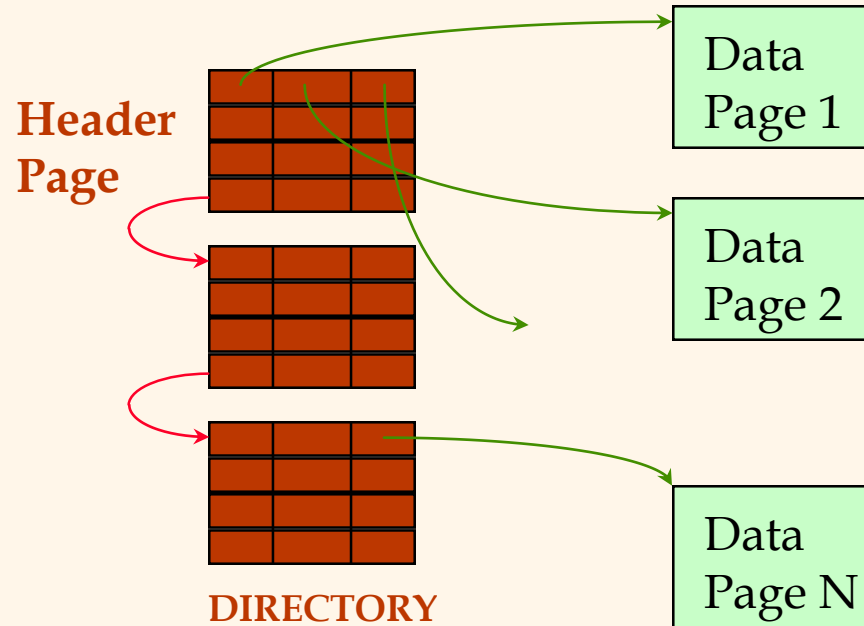
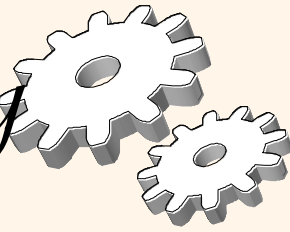
- ❖ Simplest file structure contains records in no particular order.
- ❖ As file grows and shrinks, disk pages are allocated and de-allocated.
- ❖ To support record level operations, we must:
 - keep track of the *pages* in a file
 - keep track of *free space* on pages
 - keep track of the *records* on a page
- ❖ There are many alternatives for keeping track of this.

Heap File Implemented as a List

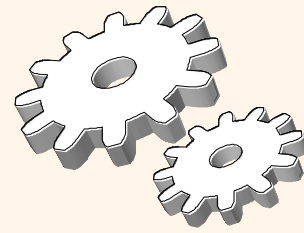


- ❖ The header page id and Heap file name must be stored someplace.
- ❖ Each page contains 2 'pointers' plus data.

Heap File Using a Page Directory

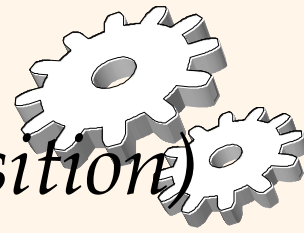


- ❖ The entry for a page can include the number of free bytes on the page.
- ❖ The directory is a collection of pages; linked list implementation is just one alternative.
 - *Much smaller than linked list of all HF pages!*



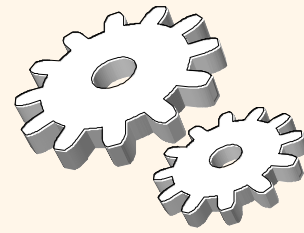
System Catalogs

- ❖ For each index:
 - structure (e.g., B+ tree) and search key fields
- ❖ For each relation:
 - name, file name, file structure (e.g., Heap file)
 - attribute name and type, for each attribute
 - index name, for each index
 - integrity constraints
- ❖ For each view:
 - view name and definition
- ❖ Plus statistics, authorization, buffer pool size, etc.
 - * *Catalogs are themselves stored as relations!*



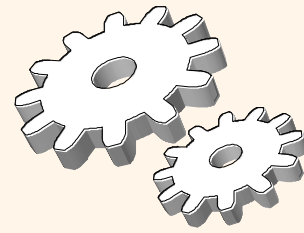
Attr_Cat(attr_name, rel_name, type, position)

attr_name	rel_name	type	position
attr_name	Attribute_Cat	string	1
rel_name	Attribute_Cat	string	2
type	Attribute_Cat	string	3
position	Attribute_Cat	integer	4
sid	Students	string	1
name	Students	string	2
login	Students	string	3
age	Students	integer	4
gpa	Students	real	5
fid	Faculty	string	1
fname	Faculty	string	2
sal	Faculty	real	3



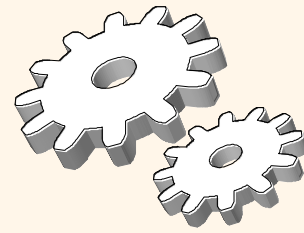
Summary

- ❖ Disks provide cheap, non-volatile storage.
 - Random access, but cost depends on location of page on disk; important to arrange data sequentially to minimize *seek* and *rotation* delays.
- ❖ Buffer manager brings pages into RAM.
 - Page stays in RAM until released by requestor.
 - Written to disk when frame chosen for replacement (which is sometime after requestor releases the page).
 - Choice of frame to replace based on *replacement policy*.
 - Tries to *pre-fetch* several pages at a time.



Summary (Contd.)

- ❖ DBMS vs. OS File Support
 - DBMS needs features not found in many OS's, e.g., forcing a page to disk, controlling the order of page writes to disk, files spanning disks, ability to control pre-fetching and page replacement policy based on predictable access patterns, etc.
- ❖ Variable length record format with field offset directory offers support for direct access to i'th field and null values.
- ❖ Slotted page format supports variable length records and allows records to move on page.



Summary (Contd.)

- ❖ File layer keeps track of pages in a file, and supports abstraction of a collection of records.
 - Pages with free space identified using linked list or directory structure (similar to how pages in file are kept track of).
- ❖ Indexes support efficient retrieval of records based on the values in some fields.
- ❖ Catalog relations store information about relations, indexes and views. (*Information that is common to all records in a given collection.*)